

Cos'è vuol dire computare

Computare significa ridurre un'informazione da una forma implicita ad una forma esplicita in modo effettivo

→ Rielaborazione informazioni → Esplicitare

Definizione di modello computazionale

Lo possiamo definire come un modello o formalismo matematico, in cui esso viene rappresentato in modo astratto e rappresentiamo un modo di definire un input, un output, un flusso e come organizzare i flussi degli elementi di computazione durante un **processo effettivo** (effettivo) che permette di rendere esplicito ciò che era implicito in partenza

Diversi modelli computazionali

- Funzioni ricorsive
- Lambda-Calcolo
- Macchine di Turing (automi con la massima potenza computazionale)

Teorie dei linguaggi: Formali

Gli ASFD riconoscono le stringhe appartenenti a linguaggi regolari

- Non possono memorizzare informazioni e parte le stringhe di input

Analisi di Alan Turing $\Rightarrow$ concetto di procedure effettive
Turing provò a formalizzare le cose di tutte le procedure effettive (1936) e questo rappresenta la 1^a analisi che descrive come ha luogo la computazione

Le configurazioni

Prendiamo 2 configurazioni C_i e C_j e indichiamo con $C_i \xrightarrow{R} C_j$ la relazione di S tramite una relazione di transizione, vale a dire C_j deriva da C_i per effetto dell'applicazione della funzione di transizione di R . ovviamente se R è totale e cui ci si riferisce è ben definito possiamo scrivere $C_i \vdash C_j$

Se prendiamo e consideriamo una sequenza di configurazioni $C_0, \dots, C_n$ che ha lunghezza finita $\Rightarrow C_n \xrightarrow{R} \textcircled{C}$ configurazione finale
 $\downarrow$

terminare in uno stato di accettazione o rifiuto

Quando la configurazione ha lunghezza finita ed è terminale esse termina

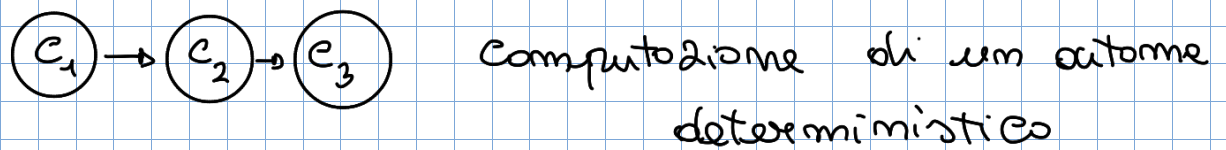
tipi di configurazione

Accettazione: situazione in cui l'automa, letta tutte le stringhe di input si trova in uno stato q finale ($q \in F$)

Rifiuto / non accettazione: situazione in cui l'automa non ha letto tutte le stringhe di input oppure se l'ha letta, non si trova in uno stato finale

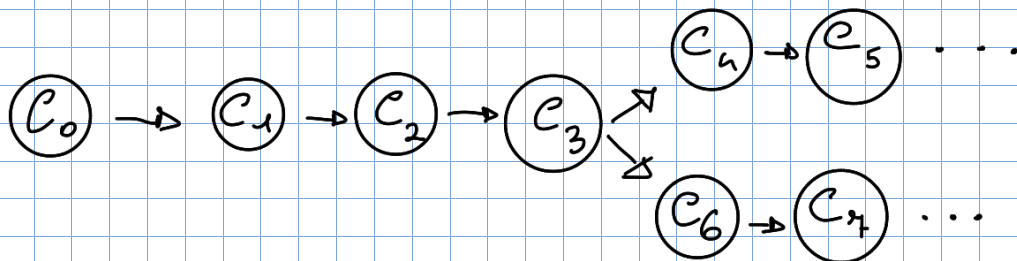
2.5.2 automi deterministici e non

Definizione: Un autome è detto **deterministico** se ogni stringa di input associa una computazione e quindi una singola sequenza di configurazioni. Possiamo dire che un autome deterministico, data una stringa n input può eseguire una sola computazione: se la computazione termina in una configurazione di accettazione, allora la stringa viene accettata.



Definizione: un autome è non deterministico se una stessa stringa di input un numero qualunque ($n > 1$) di computazioni. Osserviamo che l'autome deterministico è un caso particolare del non det per $n = 1$ non det $\Leftrightarrow$ det

Indicheremo con grado di non determinismo di un autome il numero massimo di computazione che la funzione di transizione associa ad una configurazione

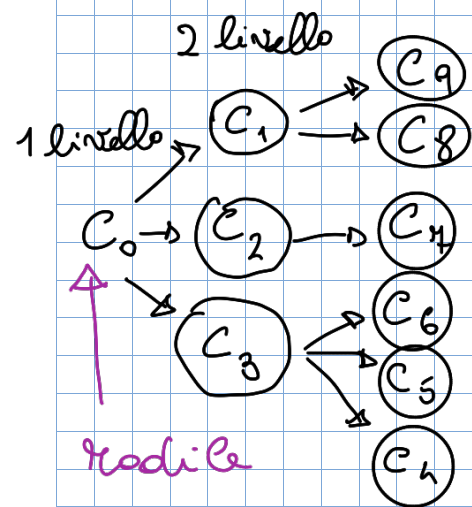


Abbiamo diverse continuazioni di una stessa computazione di un autome non deterministico

Definizione

Un automa è detto accettazione e rifiuto di una stringa se det e non det. Infatti nel caso delle non det la stringa viene accettata se una qualsunque delle computazioni definite è di accettazione, mentre non lo è se tutte le possibili computazioni che terminano non sono di accettazione.

Per un automa non deterministico, consideriamo come se l'automato esegua una sola computazione non deterministica per la quale, ad ogni passo, emerge non una sola configurazione ma un insieme di configurazioni, transitando ad ogni passo, non da una conf. a un'altra conf. ma da un insieme di config. ad un insieme di config.



Albero di Computazione

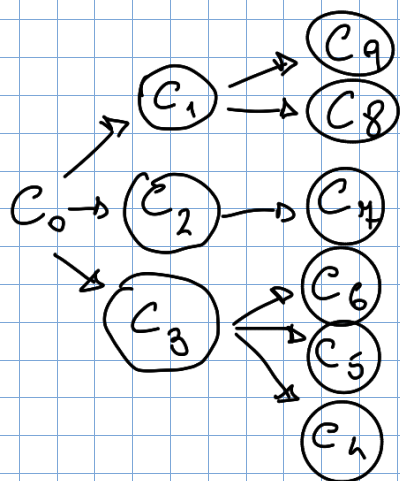
C_0, C_1, C_9

C_0, C_1, C_8

Con un albero di computazione definisco "Simple" computazioni aventi come prefissi le sequenze di configurazioni: C_0, C_3, C_4 ; C_0, C_1, C_8 ; C_0, C_3, C_6 corrispondenti ai cammini con origine nella radice dell'albero.

Il concetto di non determinismo non deve essere confuso con la definizione reale, perché questo concetto è sostanzialmente un artificio matematico e ci

Consente di rappresentare un calcolo, invece di una traiettoria definite in uno spazio di stati come un albero di traiettorie. Ciò che ci interessa è poter definire un concetto di accettazione legato al fatto che almeno uno dei rami dell'albero conduce ad uno stato finale



Espressioni regolari $\rightarrow$ descrivono tutti i linguaggi appartenenti a una classe

Sia dato un alfabeto Σ e dato l'insieme di simboli: $\{+, *, (,), \cdot, \emptyset\}$

Si definisce espressione regolare sull'alfabeto Σ una stringa

$$r \in (\Sigma \cup \{+, *, (,), \cdot, \emptyset\})^+$$

tale che valga una delle seguenti condizioni:

1. $r = \emptyset$
2. $r \in \Sigma$
3. $r = (s + t)$, oppure $r = (s \cdot t)$ oppure $r = s^*$
dove s e t sono espressioni regolari sull'alfabeto Σ

Corrispondenze tra le espressioni regolari e i linguaggi:

ESP. REG	LINGUAGGIO
$\emptyset$	$\wedge$
a	$\{a\}$
$(S+T)$	$L(S) \cup L(T)$
$(S \cdot T)$	$L(S) \circ L(T)$
S^*	$(L(S))^*$

con $L(\pi)$ generico denoto il linguaggio rappresentato dall'espressione regolare π

→ Se s e t sono due espressioni regolari sono scorciatoie (st) anziché $(s \cdot t)$

→ Diamo precedenza al simbolo $*$ sul simbolo $\cdot$,
 • la precedenza su $+$ e possiamo tenere conto delle proprietà associative di tali applicazioni

ES:

l'espressione regolare $(a + (b \cdot (c \cdot d)))$ definita sull'alfabeto $\Sigma = \{a, b, c, d\}$ può essere riscritta $(a + (b(cd))) = a + bcd$

anche per espressioni regolari, come per i simboli corollari o stringhe possiamo introdurre le potenze scrivendo $(\pi)^3 \rightarrow \underbrace{\pi \pi \pi}_{\text{espressione regolare}}$

la chiusura non riflessiva π^+ per indicare $\pi(\pi)^*$

Observazione

Per rappresentare le stringhe vuote può essere utile a volte usare il simbolo ϵ nella

espressioni regolari, con lo scopo di indicare
il linguaggio $\{E\}$

$$\Lambda = \{E\}$$